

05_Fandango

July 13, 2025

```
[528]: import numpy as np
import matplotlib.pyplot as plt
import pandas as pd
import seaborn as sns
```

```
[529]: import warnings
warnings.filterwarnings('ignore')
```

```
[530]: fandango = pd.read_csv('fandango_scrape.csv')
fandango
```

```
[530]:
```

	FILM	STARS	RATING	VOTES
0	Fifty Shades of Grey (2015)	4.0	3.9	34846
1	Jurassic World (2015)	4.5	4.5	34390
2	American Sniper (2015)	5.0	4.8	34085
3	Furious 7 (2015)	5.0	4.8	33538
4	Inside Out (2015)	4.5	4.5	15749
..	...	...	...	...
499	Valiyavan (2015)	0.0	0.0	0
500	WWE SummerSlam 2015 (2015)	0.0	0.0	0
501	Yagavarayinum Naa Kaakka (2015)	0.0	0.0	0
502	Yesterday, Today and Tomorrow (1964)	0.0	0.0	0
503	Zarafa (2012)	0.0	0.0	0

[504 rows x 4 columns]

```
[531]: fandango.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 504 entries, 0 to 503
Data columns (total 4 columns):
#   Column  Non-Null Count  Dtype
---  -
0    FILM    504 non-null      object
1    STARS    504 non-null      float64
2    RATING   504 non-null      float64
3    VOTES    504 non-null      int64
```

```
dtypes: float64(2), int64(1), object(1)
memory usage: 15.9+ KB
```

```
[532]: fandango.describe()
```

```
[532]:
```

	STARS	RATING	VOTES
count	504.000000	504.000000	504.000000
mean	3.558532	3.375794	1147.863095
std	1.563133	1.491223	3830.583136
min	0.000000	0.000000	0.000000
25%	3.500000	3.100000	3.000000
50%	4.000000	3.800000	18.500000
75%	4.500000	4.300000	189.750000
max	5.000000	5.000000	34846.000000

0.0.1 Rating vs Votes plot (Popularity)

- Popularity
- A high number of votes generally indicates the movie is widely watched or popular.
- A low vote count might mean the movie is lesser-known or recently released.

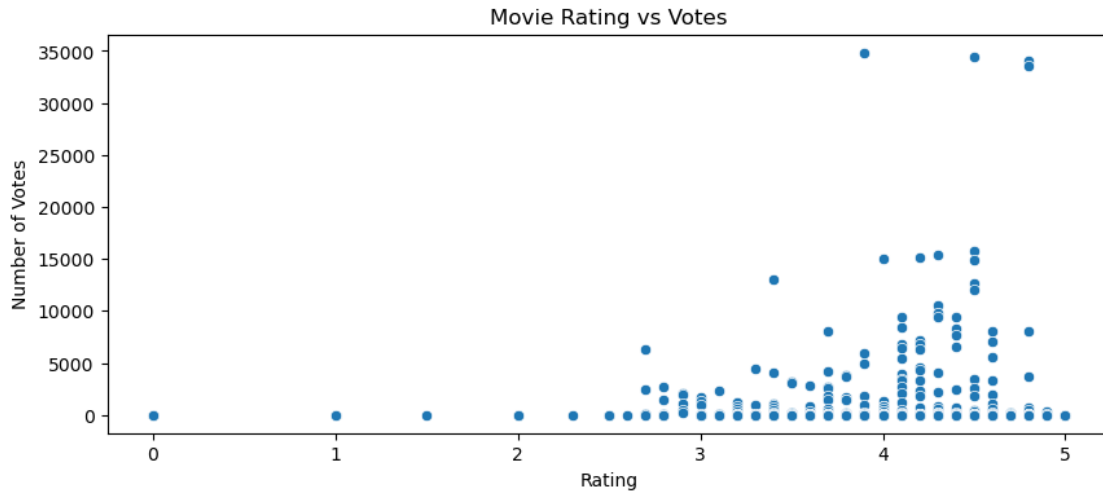
```
[533]: fandango.head()
```

```
[533]:
```

	FILM	STARS	RATING	VOTES
0	Fifty Shades of Grey (2015)	4.0	3.9	34846
1	Jurassic World (2015)	4.5	4.5	34390
2	American Sniper (2015)	5.0	4.8	34085
3	Furious 7 (2015)	5.0	4.8	33538
4	Inside Out (2015)	4.5	4.5	15749

```
[534]: #fandango_clean = fandango.dropna(subset=['RATINGS', 'VOTES']) We have seen no
      ↪null values in data set
      #sns.scatterplot(data = fandango_clean, x = 'RATINGS', y = 'VOTES')
```

```
[535]: plt.figure(figsize=(10, 4))
      sns.scatterplot(data=fandango, x='RATING', y='VOTES')
      plt.title('Movie Rating vs Votes')
      plt.xlabel('Rating')
      plt.ylabel('Number of Votes')
      plt.show()
```



```
[536]: # Understand the correlation:

# Only select numeric columns
numeric_df = fandango.select_dtypes(include='number')

numeric_df
```

```
[536]:
```

	STARS	RATING	VOTES
0	4.0	3.9	34846
1	4.5	4.5	34390
2	5.0	4.8	34085
3	5.0	4.8	33538
4	4.5	4.5	15749
..	...	...	...
499	0.0	0.0	0
500	0.0	0.0	0
501	0.0	0.0	0
502	0.0	0.0	0
503	0.0	0.0	0

[504 rows x 3 columns]

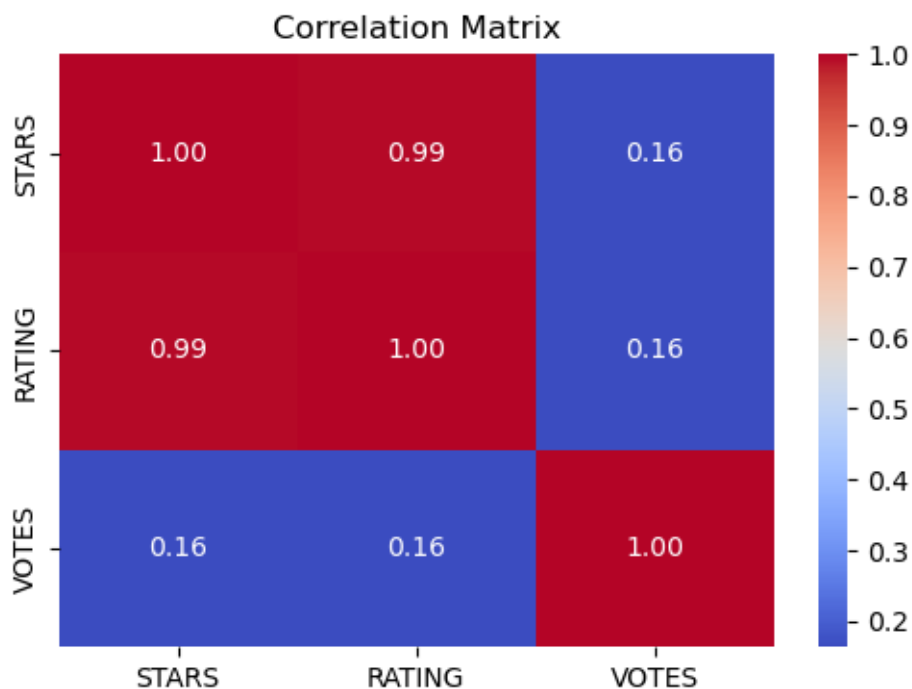
```
[537]: # Compute correlation
corr_matrix = numeric_df.corr()
corr_matrix
```

```
[537]:
```

	STARS	RATING	VOTES
STARS	1.000000	0.994696	0.164218
RATING	0.994696	1.000000	0.163764
VOTES	0.164218	0.163764	1.000000

```
[538]: # Optional: display it as a heatmap
```

```
plt.figure(figsize=(6, 4))
sns.heatmap(corr_matrix, annot=True, cmap='coolwarm', fmt=".2f")
plt.title("Correlation Matrix")
plt.show()
```



```
[539]: fandango.head()
```

```
[539]:
```

	FILM	STARS	RATING	VOTES
0	Fifty Shades of Grey (2015)	4.0	3.9	34846
1	Jurassic World (2015)	4.5	4.5	34390
2	American Sniper (2015)	5.0	4.8	34085
3	Furious 7 (2015)	5.0	4.8	33538
4	Inside Out (2015)	4.5	4.5	15749

1. Pearson Correlation

Measures: Linear relationship between two continuous variables.

Assumes: Both variables are normally distributed and have a linear relationship.

Values range: From -1 to 1.

+1: Perfect positive linear correlation.

0: No linear correlation.

-1: Perfect negative linear correlation.

Example Use Case: “Does a higher rating always mean more votes?”

```
[540]: fandango.corr(method='pearson')
```

```
[540]:
```

	STARS	RATING	VOTES
STARS	1.000000	0.994696	0.164218
RATING	0.994696	1.000000	0.163764
VOTES	0.164218	0.163764	1.000000

2. Spearman Rank Correlation

Measures: Monotonic relationship (not necessarily linear).

Assumes: The relationship may be non-linear, but ordered (i.e., as one variable increases, the other tends to increase or decrease).

Use when: Your data is not normally distributed, contains outliers, or you’re dealing with ranks or ordinal data.

Internally, it ranks the values and applies Pearson correlation on the ranks.

```
[541]: fandango.corr(method='spearman')
```

```
[541]:
```

	STARS	RATING	VOTES
STARS	1.000000	0.980682	0.413909
RATING	0.980682	1.000000	0.379608
VOTES	0.413909	0.379608	1.000000

```
[542]: fandango.corr() # by default pandas data frame uses : pearson
```

```
[542]:
```

	STARS	RATING	VOTES
STARS	1.000000	0.994696	0.164218
RATING	0.994696	1.000000	0.163764
VOTES	0.164218	0.163764	1.000000

0.0.2 When to use what?

Situation	Use
Variables are continuous and linearly related	Pearson
Data is ordinal or has monotonic but non-linear trend	Spearman
Data has outliers or not normally distributed	Prefer Spearman

```
[543]: # default correlation
fandango.select_dtypes(include='number').corr()
```

```
[543]:
```

	STARS	RATING	VOTES
STARS	1.000000	0.994696	0.164218
RATING	0.994696	1.000000	0.163764
VOTES	0.164218	0.163764	1.000000

```
[544]: # Pearson correlation
fandango.select_dtypes(include='number').corr(method='pearson')
```

```
[544]:
```

	STARS	RATING	VOTES
STARS	1.000000	0.994696	0.164218
RATING	0.994696	1.000000	0.163764
VOTES	0.164218	0.163764	1.000000

```
[545]: # Spearman correlation
fandango.select_dtypes(include='number').corr(method='spearman')
```

```
[545]:
```

	STARS	RATING	VOTES
STARS	1.000000	0.980682	0.413909
RATING	0.980682	1.000000	0.379608
VOTES	0.413909	0.379608	1.000000

0.0.3 Extract the years from the movie name:

```
[546]: fandango['FILM']
```

```
[546]: 0          Fifty Shades of Grey (2015)
1          Jurassic World (2015)
2          American Sniper (2015)
3          Furious 7 (2015)
4          Inside Out (2015)
...
499          Valiyavan (2015)
500          WWE SummerSlam 2015 (2015)
501          Yagavarayinum Naa Kaakka (2015)
502          Yesterday, Today and Tomorrow (1964)
503          Zarafa (2012)
Name: FILM, Length: 504, dtype: object
```

```
[547]: fandango['YEAR'] = fandango['FILM'].apply(lambda title : title[-5:-1])
```

```
[548]: fandango.head()
```

```
[548]:
```

	FILM	STARS	RATING	VOTES	YEAR
0	Fifty Shades of Grey (2015)	4.0	3.9	34846	2015
1	Jurassic World (2015)	4.5	4.5	34390	2015
2	American Sniper (2015)	5.0	4.8	34085	2015
3	Furious 7 (2015)	5.0	4.8	33538	2015

4	Inside Out (2015)	4.5	4.5	15749	2015
---	-------------------	-----	-----	-------	------

```
[549]: fandango['YEAR'].value_counts()
```

```
[549]: 2015    478
      2014     23
      2016      1
      1964      1
      2012      1
      Name: YEAR, dtype: int64
```

Top 10 movies with highest number of votes

```
[550]: fandango.nlargest(10, 'VOTES')
```

```
[550]:
```

		FILM	STARS	RATING	VOTES	\
0		Fifty Shades of Grey (2015)	4.0	3.9	34846	
1		Jurassic World (2015)	4.5	4.5	34390	
2		American Sniper (2015)	5.0	4.8	34085	
3		Furious 7 (2015)	5.0	4.8	33538	
4		Inside Out (2015)	4.5	4.5	15749	
5	The Hobbit: The Battle of the Five Armies (2014)		4.5	4.3	15337	
6	Kingsman: The Secret Service (2015)		4.5	4.2	15205	
7	Minions (2015)		4.0	4.0	14998	
8	Avengers: Age of Ultron (2015)		5.0	4.5	14846	
9	Into the Woods (2014)		3.5	3.4	13055	


```
YEAR
```

0	2015
1	2015
2	2015
3	2015
4	2015
5	2014
6	2015
7	2015
8	2015
9	2014

```
[551]: fandango.head()
```

```
[551]:
```

		FILM	STARS	RATING	VOTES	YEAR
0	Fifty Shades of Grey (2015)		4.0	3.9	34846	2015
1	Jurassic World (2015)		4.5	4.5	34390	2015
2	American Sniper (2015)		5.0	4.8	34085	2015
3	Furious 7 (2015)		5.0	4.8	33538	2015
4	Inside Out (2015)		4.5	4.5	15749	2015

```
[552]: # Other way
# Sort by 'VOTES' column in descending order and select top 10
top10_votes = fandango.sort_values(by='VOTES', ascending=False).head(10)

# Display the result
print(top10_votes[['FILM', 'VOTES']])
```

	FILM	VOTES
0	Fifty Shades of Grey (2015)	34846
1	Jurassic World (2015)	34390
2	American Sniper (2015)	34085
3	Furious 7 (2015)	33538
4	Inside Out (2015)	15749
5	The Hobbit: The Battle of the Five Armies (2014)	15337
6	Kingsman: The Secret Service (2015)	15205
7	Minions (2015)	14998
8	Avengers: Age of Ultron (2015)	14846
9	Into the Woods (2014)	13055

```
[553]: # Other way by groupby: but if data have movies with same name multiple times
        ↳ it will avg out:
top10_votes = (
    fandango.groupby('FILM')['VOTES']
    .sum() # or .max(), .mean(), etc.
    .sort_values(ascending=False)
    .head(10)
)

print(top10_votes)
```

FILM	VOTES
Fifty Shades of Grey (2015)	34846
Jurassic World (2015)	34390
American Sniper (2015)	34085
Furious 7 (2015)	33538
Inside Out (2015)	15749
The Hobbit: The Battle of the Five Armies (2014)	15337
Kingsman: The Secret Service (2015)	15205
Minions (2015)	14998
Avengers: Age of Ultron (2015)	14846
Into the Woods (2014)	13055

Name: VOTES, dtype: int64

How many movies with zero votes?

```
[554]: (fandango['VOTES'] == 0).sum()
```

[554]: 69


```
[555]: # Name of those movies:
fandango[fandango['VOTES'] == 0]
```

```
[555]:
```

	FILM	STARS	RATING	VOTES	YEAR
435	6 Years (2015)	0.0	0.0	0	2015
436	7 Minutes (2015)	0.0	0.0	0	2015
437	A Year in Champagne (2015)	0.0	0.0	0	2015
438	Balls Out (2015)	0.0	0.0	0	2015
439	Before I Wake (2015)	0.0	0.0	0	2015
..	...	...	...	...	
499	Valiyavan (2015)	0.0	0.0	0	2015
500	WWE SummerSlam 2015 (2015)	0.0	0.0	0	2015
501	Yagavarayinum Naa Kaakka (2015)	0.0	0.0	0	2015
502	Yesterday, Today and Tomorrow (1964)	0.0	0.0	0	1964
503	Zarafa (2012)	0.0	0.0	0	2012

[69 rows x 5 columns]

```
[556]: # Remove those movies with zero ratings from the data set:
fandango[fandango['VOTES'] > 0]
```

```
[556]:
```

	FILM	STARS	RATING	VOTES	YEAR
0	Fifty Shades of Grey (2015)	4.0	3.9	34846	2015
1	Jurassic World (2015)	4.5	4.5	34390	2015
2	American Sniper (2015)	5.0	4.8	34085	2015
3	Furious 7 (2015)	5.0	4.8	33538	2015
4	Inside Out (2015)	4.5	4.5	15749	2015
..	...	...	...	...	
430	That Sugar Film (2015)	5.0	5.0	1	2015
431	The Intern (2015)	5.0	5.0	1	2015
432	The Park Bench (2015)	5.0	5.0	1	2015
433	The Wanted 18 (2015)	5.0	5.0	1	2015
434	Z For Zachariah (2015)	5.0	5.0	1	2015

[435 rows x 5 columns]

```
[557]: fan_reviewed = fandango[fandango['VOTES'] > 0]
fan_reviewed
```

```
[557]:
```

	FILM	STARS	RATING	VOTES	YEAR
0	Fifty Shades of Grey (2015)	4.0	3.9	34846	2015
1	Jurassic World (2015)	4.5	4.5	34390	2015
2	American Sniper (2015)	5.0	4.8	34085	2015
3	Furious 7 (2015)	5.0	4.8	33538	2015
4	Inside Out (2015)	4.5	4.5	15749	2015
..	...	...	...	...	
430	That Sugar Film (2015)	5.0	5.0	1	2015

431	The Intern (2015)	5.0	5.0	1	2015
432	The Park Bench (2015)	5.0	5.0	1	2015
433	The Wanted 18 (2015)	5.0	5.0	1	2015
434	Z For Zachariah (2015)	5.0	5.0	1	2015

[435 rows x 5 columns]

```
[558]: # To get the genral info
fan_reviewed.info()
fan_reviewed.describe()
fan_reviewed.head()
```

```
<class 'pandas.core.frame.DataFrame'>
Int64Index: 435 entries, 0 to 434
Data columns (total 5 columns):
 #   Column  Non-Null Count  Dtype
---  -
 0   FILM    435 non-null    object
 1   STARS    435 non-null    float64
 2   RATING   435 non-null    float64
 3   VOTES    435 non-null    int64
 4   YEAR     435 non-null    object
dtypes: float64(2), int64(1), object(2)
memory usage: 20.4+ KB
```

```
[558]:
```

		FILM	STARS	RATING	VOTES	YEAR
0	Fifty Shades of Grey (2015)		4.0	3.9	34846	2015
1	Jurassic World (2015)		4.5	4.5	34390	2015
2	American Sniper (2015)		5.0	4.8	34085	2015
3	Furious 7 (2015)		5.0	4.8	33538	2015
4	Inside Out (2015)		4.5	4.5	15749	2015

0.0.4 Background: Fandango Rating Controversy

- Fandango showed **star ratings** (e.g., 4.5 stars out of 5) prominently on their website.
- However, the actual **average user ratings** (e.g., 4.2 or 3.9) were often **lower**.
- Fandango was **rounding up aggressively**, which made movies appear better rated than they really were.

Examples:

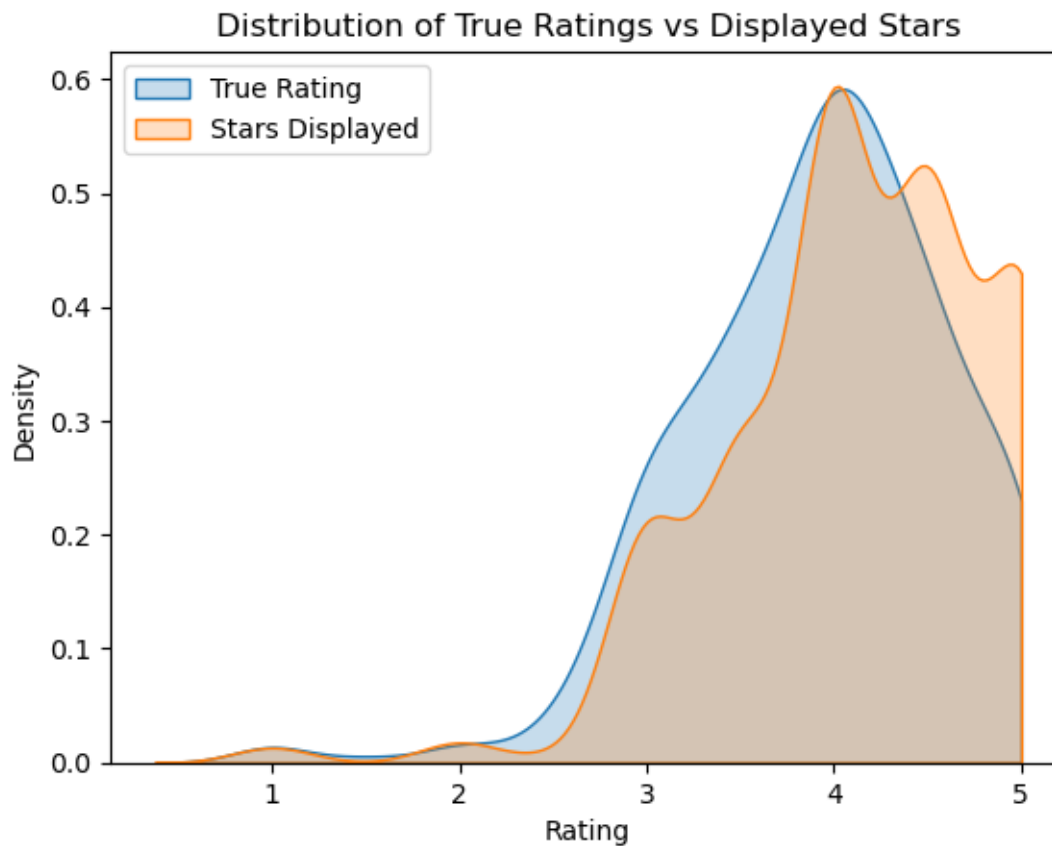
- A **4.1** average might be displayed as **4.5 stars**
- A **3.6** average might be shown as **4 stars**

This gave the impression that movies were rated higher than they truly were.

This issue was exposed in a well-known data journalism article by **Walt Hickey** at [FiveThirtyEight](#).

```
[559]: sns.kdeplot(data=fan_reviewed, x='RATING', fill=True, label='True Rating',
↳ clip=[0, 5])
sns.kdeplot(data=fan_reviewed, x='STARS', fill=True, label='Stars Displayed',
↳ clip=[0, 5])

plt.title("Distribution of True Ratings vs Displayed Stars")
plt.xlabel("Rating")
plt.ylabel("Density")
plt.legend()
plt.show()
```



To quantify the discrepancy

```
[560]: fan_reviewed = fan_reviewed.copy()
fan_reviewed['STARS_DIFF'] = np.round(fan_reviewed['STARS'] -
↳ fan_reviewed['RATING'],2)
fan_reviewed.head()
```

```
[560]:
```

	FILM	STARS	RATING	VOTES	YEAR	STARS_DIFF
0	Fifty Shades of Grey (2015)	4.0	3.9	34846	2015	0.1

1	Jurassic World (2015)	4.5	4.5	34390	2015	0.0
2	American Sniper (2015)	5.0	4.8	34085	2015	0.2
3	Furious 7 (2015)	5.0	4.8	33538	2015	0.2
4	Inside Out (2015)	4.5	4.5	15749	2015	0.0

0.0.5 Understanding STARS_DIFF (STARS - RATING)

The column STARS_DIFF represents the difference between the number of stars Fandango **displayed** for a movie and the actual **average user rating**.

What does this difference mean?

STARS_DIFF Value	Interpretation
0.0	The displayed stars exactly match the average rating
> 0.0	The displayed stars are higher than the actual rating (<i>Inflated</i>)
< 0.0	The displayed stars are lower than the actual rating (rare)

Why is this important? This difference reveals potential **bias** or **manipulation** in how Fandango presented ratings:

- Inflated star ratings can **mislead viewers**, making a movie appear better than it actually is.
 - For example:
 - A movie with an actual rating of **3.6** might have been shown as **4.0 stars** (STARS_DIFF = 0.4).
 - Users may trust the visual star rating more than the numeric rating.
-

What does the countplot of STARS_DIFF show?

- It shows **how many movies** had each level of inflation.
 - A high count of movies with 0.2, 0.3, or 0.5 difference indicates that Fandango often **rounded up** ratings.
 - This supports the claim that Fandango's rating display was **systematically biased upward**.
-

This issue was originally uncovered in a 2015 data journalism article by **Walt Hickey** on [FiveThirtyEight](#).

0.0.6 Example: Understanding STARS_DIFF with a Single Movie

Let's take this example from the dataset:

FILM	STARS	RATING	VOTES
<i>Fifty Shades of Grey (2015)</i>	4.0	3.9	34846

What does this mean?

- **STARS:** 4.0 → This is what Fandango **displayed** on their website.
- **RATING:** 3.9 → This is the **actual average user rating**.
- **Difference:** STARS_DIFF = 4.0 - 3.9 = 0.1

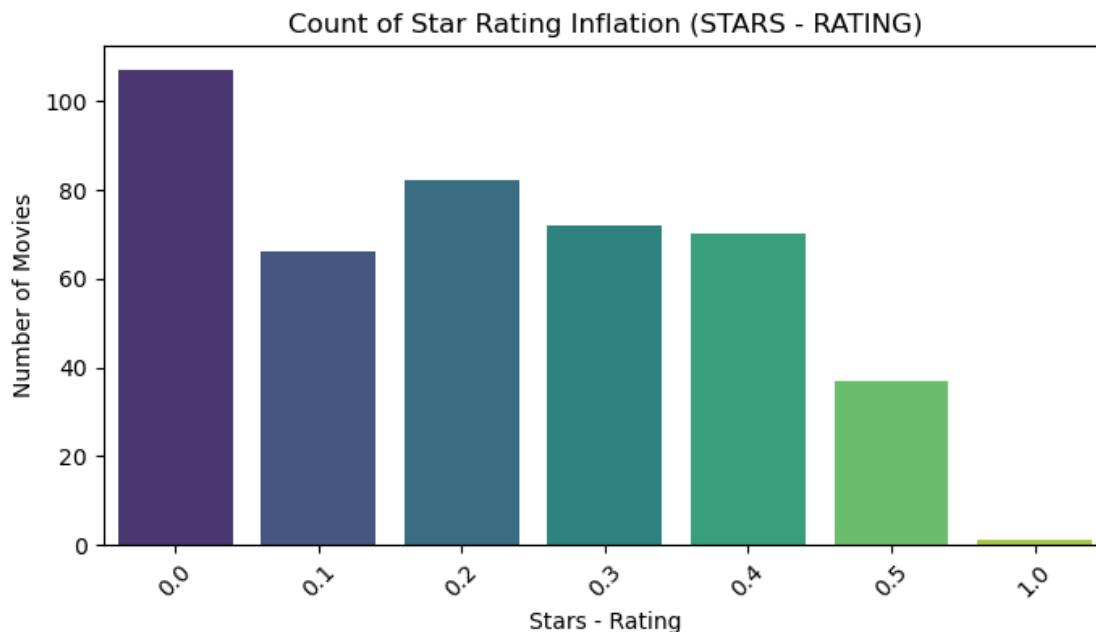
Interpretation: Fandango showed **4 stars**, even though users rated it **3.9** on average.

This is a small difference, but when it happens **systematically across many movies**, it can **bias user perception** — making movies appear better than they are.

The STARS_DIFF helps us **quantify** how often and by how much Fandango **inflated** ratings compared to actual user reviews.

How many time or for how many movies this same discrepancy occurs

```
[561]: plt.figure(figsize=(8, 4))
sns.countplot(data=fan_reviewed, x='STARS_DIFF', palette='viridis') # or
↳ 'mako', 'Set2', 'pastel', etc.
plt.xticks(rotation=45)
plt.title('Count of Star Rating Inflation (STARS - RATING)')
plt.xlabel('Stars - Rating')
plt.ylabel('Number of Movies')
plt.show()
```



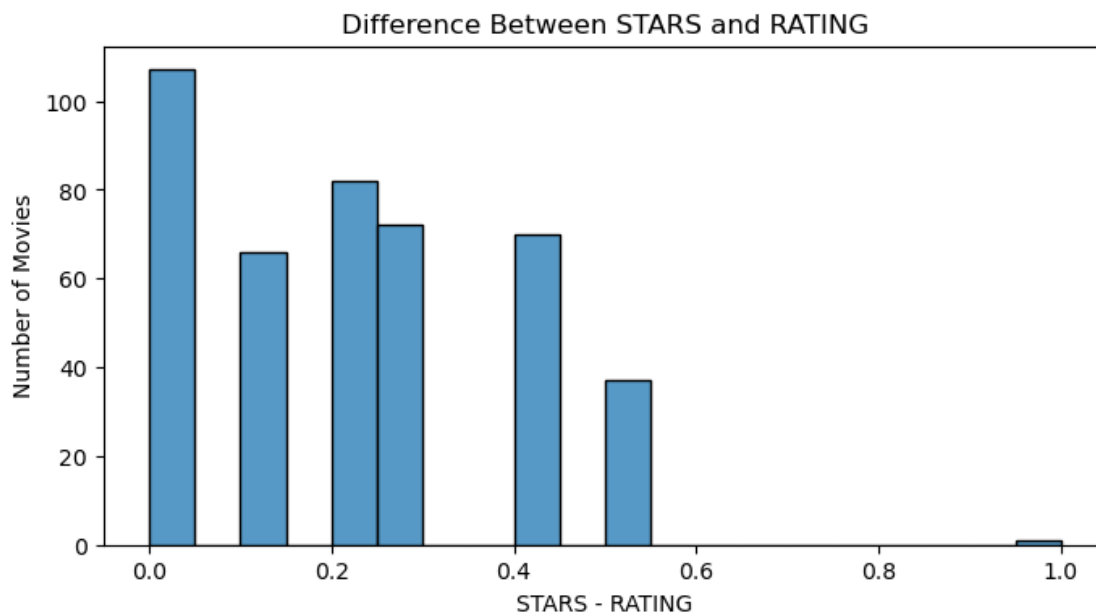
```
[562]: fan_reviewed.head()
```

```
[562]:
```

	FILM	STARS	RATING	VOTES	YEAR	STARS_DIFF
0	Fifty Shades of Grey (2015)	4.0	3.9	34846	2015	0.1
1	Jurassic World (2015)	4.5	4.5	34390	2015	0.0
2	American Sniper (2015)	5.0	4.8	34085	2015	0.2
3	Furious 7 (2015)	5.0	4.8	33538	2015	0.2
4	Inside Out (2015)	4.5	4.5	15749	2015	0.0

```
[563]: # Other way using histo gram
```

```
plt.figure(figsize=(8,4))
sns.histplot(fan_reviewed['STARS_DIFF'], bins=20, kde=False)
plt.title("Difference Between STARS and RATING")
plt.xlabel("STARS - RATING")
plt.ylabel("Number of Movies")
plt.show()
```



Let us see what are all the movies has this highest difference as 1 (from above)

```
[564]: fan_reviewed[fan_reviewed['STARS_DIFF'] == 1]
```

```
[564]:
```

	FILM	STARS	RATING	VOTES	YEAR	STARS_DIFF
381	Turbo Kid (2015)	5.0	4.0	2	2015	1.0

0.0.7 Predicting the clean star : (Without Machine Learning)

1. Create a “clean” star rating

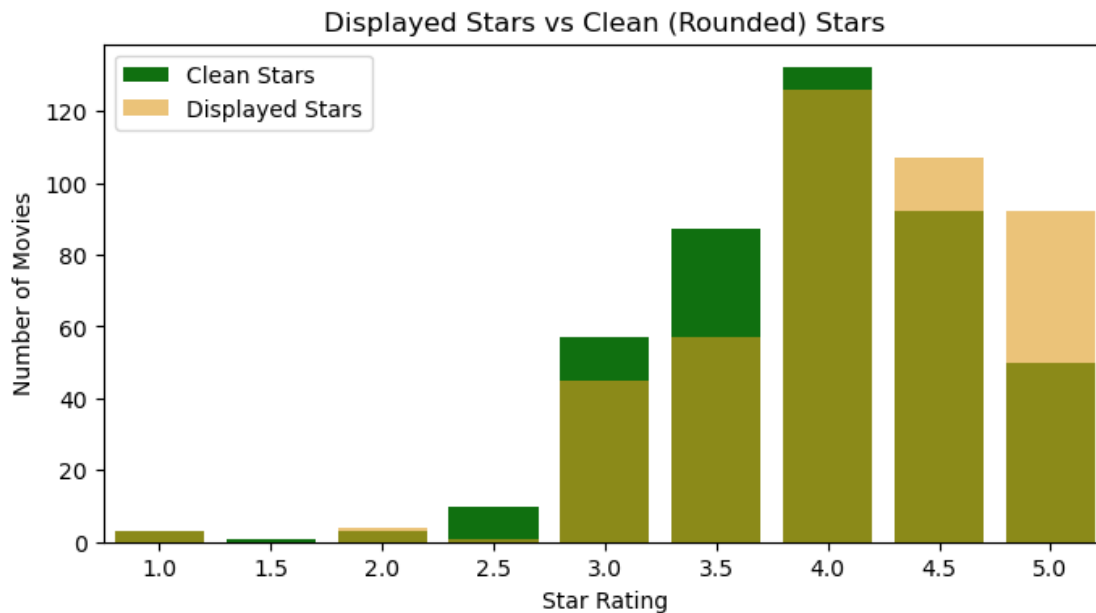
```
[565]: # Round actual rating to nearest 0.5 to simulate a fair star system
import numpy as np
fan_reviewed['CLEAN_STARS'] = np.round(fan_reviewed['RATING'] * 2) / 2

# Compare visually
fan_reviewed[['FILM', 'STARS', 'RATING', 'CLEAN_STARS']].head()
```

```
[565]:
```

	FILM	STARS	RATING	CLEAN_STARS
0	Fifty Shades of Grey (2015)	4.0	3.9	4.0
1	Jurassic World (2015)	4.5	4.5	4.5
2	American Sniper (2015)	5.0	4.8	5.0
3	Furious 7 (2015)	5.0	4.8	5.0
4	Inside Out (2015)	4.5	4.5	4.5

```
[566]: plt.figure(figsize=(8,4))
sns.countplot(data=fan_reviewed, x='CLEAN_STARS', color='green', label='Clean_
↳Stars')
sns.countplot(data=fan_reviewed, x='STARS', color='orange', alpha=0.6,
↳label='Displayed Stars')
plt.legend()
plt.title("Displayed Stars vs Clean (Rounded) Stars")
plt.xlabel("Star Rating")
plt.ylabel("Number of Movies")
plt.show()
```



```
[567]: fan_reviewed.head()
```

```
[567]:
```

	FILM	STARS	RATING	VOTES	YEAR	STARS_DIFF	\
0	Fifty Shades of Grey (2015)	4.0	3.9	34846	2015	0.1	
1	Jurassic World (2015)	4.5	4.5	34390	2015	0.0	
2	American Sniper (2015)	5.0	4.8	34085	2015	0.2	
3	Furious 7 (2015)	5.0	4.8	33538	2015	0.2	
4	Inside Out (2015)	4.5	4.5	15749	2015	0.0	

```
CLEAN_STARS
```

0	4.0
1	4.5
2	5.0
3	5.0
4	4.5

0.1 Comparing fandango ratings to other sites

```
[568]: all_sites = pd.read_csv('all_sites_scores.csv')
all_sites
```

```
[568]:
```

	FILM	RottenTomatoes	\
0	Avengers: Age of Ultron (2015)	74	
1	Cinderella (2015)	85	
2	Ant-Man (2015)	80	
3	Do You Believe? (2015)	18	


```

4          Hot Tub Time Machine 2 (2015)          14
..
141          Mr. Holmes (2015)          87
142          '71 (2015)          97
143          Two Days, One Night (2014)          97
144  Gett: The Trial of Viviane Amsalem (2015)          100
145          Kumiko, The Treasure Hunter (2015)          87

```

```

      RottenTomatoes_User  Metacritic  Metacritic_User  IMDB  \
0          86          66          7.1  7.8
1          80          67          7.5  7.1
2          90          64          8.1  7.8
3          84          22          4.7  5.4
4          28          29          3.4  5.1
..
141         78          67          7.9  7.4
142         82          83          7.5  7.2
143         78          89          8.8  7.4
144         81          90          7.3  7.8
145         63          68          6.4  6.7

```

```

      Metacritic_user_vote_count  IMDB_user_vote_count
0          1330          271107
1          249          65709
2          627          103660
3           31          3136
4           88          19560
..
141          33          7367
142          60          24116
143          123          24345
144          19          1955
145          19          5289

```

[146 rows x 8 columns]

```
[569]: all_sites.info()
```

```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 146 entries, 0 to 145
Data columns (total 8 columns):
#   Column              Non-Null Count  Dtype
---  -
0   FILM                146 non-null   object
1   RottenTomatoes      146 non-null   int64
2   RottenTomatoes_User 146 non-null   int64
3   Metacritic          146 non-null   int64

```

```

4   Metacritic_User          146 non-null    float64
5   IMDB                    146 non-null    float64
6   Metacritic_user_vote_count 146 non-null    int64
7   IMDB_user_vote_count     146 non-null    int64
dtypes: float64(2), int64(5), object(1)
memory usage: 9.2+ KB

```

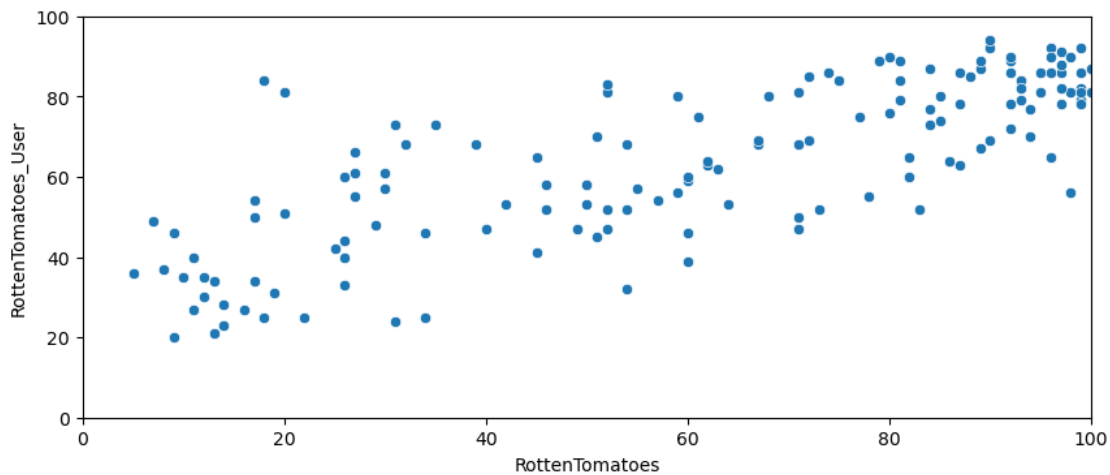
0.2 Rotten Tomatos

```

[570]: plt.figure(figsize = (10,4))
sns.scatterplot(data = all_sites, x= 'RottenTomatoes', y =_
↳'RottenTomatoes_User')
plt.xlim(0,100)
plt.ylim(0,100)

```

[570]: (0.0, 100.0)



```

[571]: # Here u can see good correlation (approx linear)

```

```

[572]: # (all_sites['RottenTomatoes'] - all_sites['RottenTomatoes_User']).apply(abs)
all_sites['Rotten_diff'] = (all_sites['RottenTomatoes'] -_
↳all_sites['RottenTomatoes_User'])
all_sites['Rotten_diff'].apply(abs).mean()

```

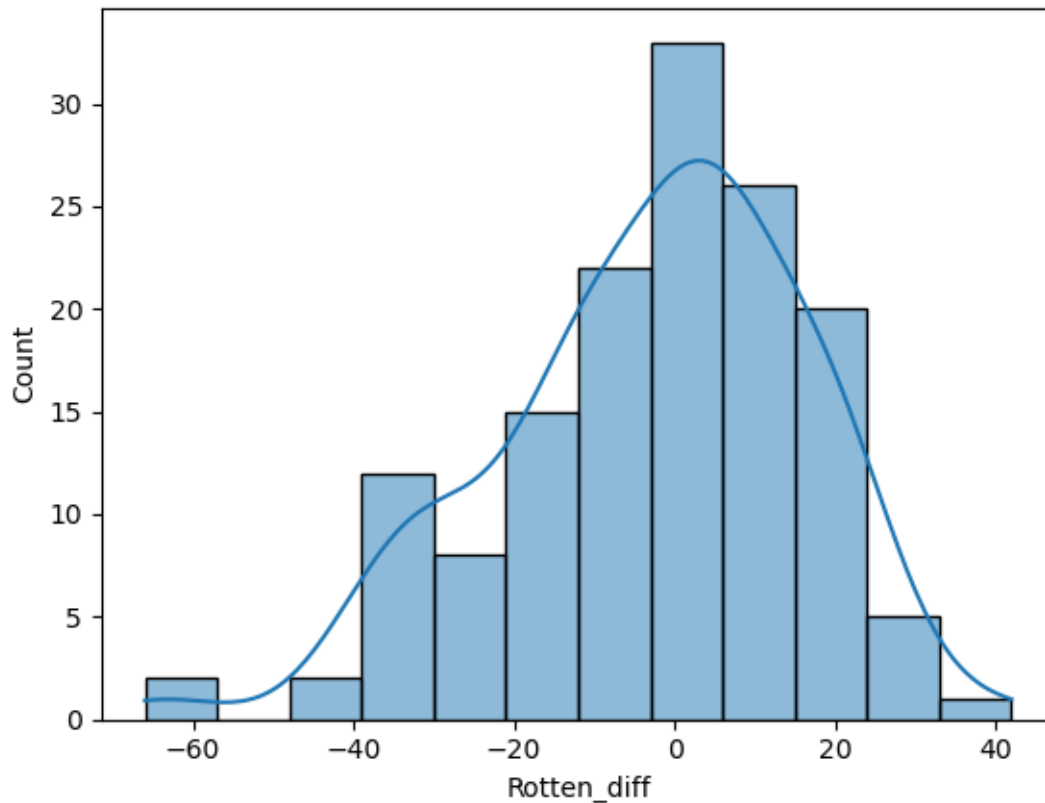
[572]: 15.095890410958905

```

[573]: sns.histplot(data = all_sites, x = 'Rotten_diff', kde = True)

```

[573]: <Axes: xlabel='Rotten_diff', ylabel='Count'>



0.2.1 What the Plot Shows

You observed a **normal-like distribution centered around 0**, which tells us:

1. Most movies have a small difference between critic and user ratings

- The mean is near 0 → Critics and users **generally agree**.
- The bell shape suggests **balanced disagreement**:
 - Some movies rated higher by critics
 - Some rated higher by users

2. Spread of the distribution (width of the curve)

- A **narrow curve** → Critics and users usually **agree closely**
- A **wider curve** → More **variation in opinions**

3. Tails (extremes)

- **Long tails** indicate strong disagreement:
 - Critics loved it, users didn't
 - Users loved it, critics didn't

0.2.2 What This Tells You

- On average, critics and users tend to **agree**
 - But there are several movies with **noticeable differences** in opinion
 - Helps detect:
 - **Rating bias**
 - **Polarization**
 - **Audience-critic mismatch**
-

0.2.3 Is This Insight Meaningful?

Absolutely! You're using **data visualization** effectively to:

- Explore the **distribution** of rating differences
- Quantify the **level of agreement** between two groups
- Identify potential **bias, controversy, or mismatches in perception**

0.2.4 Why the Bell Curve (Normal Distribution) Is Common in Natural Phenomena

The **bell curve**, also called the **normal distribution**, appears frequently in real-world data due to deep mathematical principles.

0.2.5 1. Central Limit Theorem (CLT)

If you take the average of many random, independent processes, the result tends to follow a **normal distribution** — even if the original variables themselves are not normally distributed.

Example:

Human height, test scores, or reaction time — all are influenced by **many small, random factors**. When these combine, the outcome naturally forms a bell curve.

0.2.6 2. Symmetry of Randomness

- Random variation in nature often leads to small **positive and negative changes** around a central average.
 - This creates a **symmetrical distribution**, which is exactly the shape of a bell curve.
-

0.2.7 3. Mathematical Simplicity and Power

- The normal distribution is defined by just **two parameters**:
 - Mean ()
 - Standard deviation ()
- It's mathematically elegant and predictable:
 - ~68% of values fall within ± 1
 - ~95% fall within ± 2
 - ~99.7% fall within ± 3

This makes it **very useful** in statistics, probability, and analytics.

0.2.8 4. Real-World Examples That Follow Bell Curves

Phenomenon	Why it Follows a Bell Curve
Human height	Many small genetic and environmental effects
Measurement errors	Random noise in instruments or readings
IQ scores	Designed to follow a normal distribution
Exam scores (large classes)	Influenced by preparation, sleep, difficulty, randomness
Manufacturing defects	Small random variations in production processes

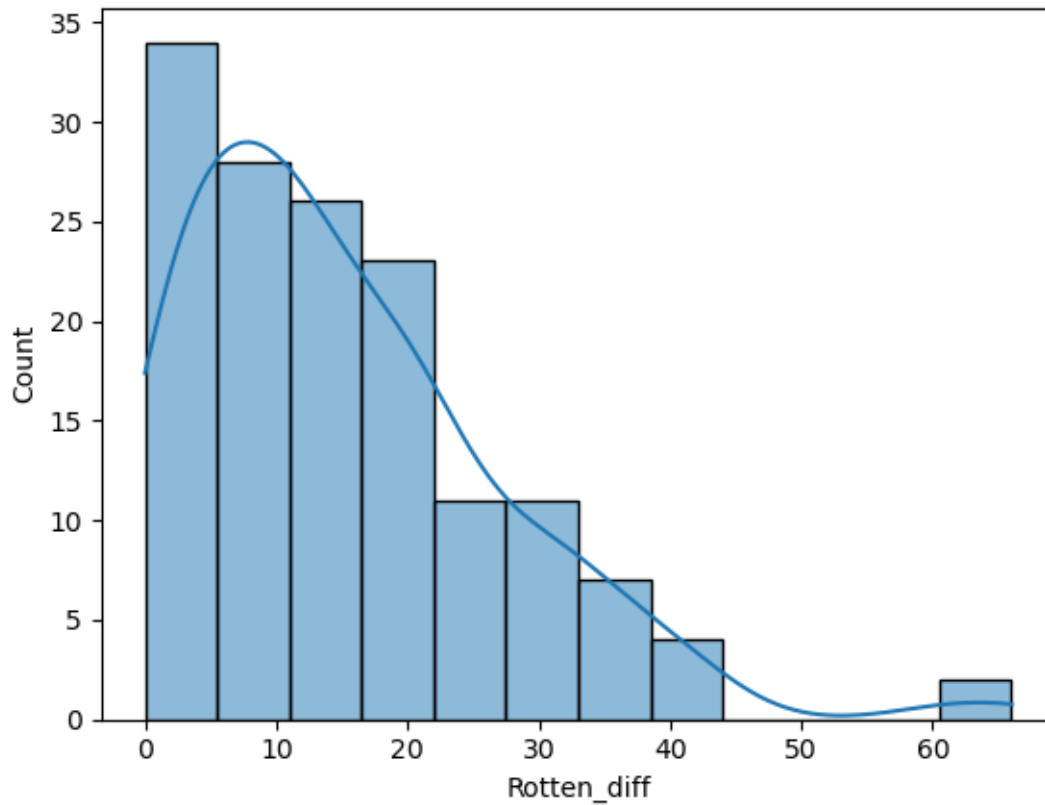
0.2.9 Summary:

The bell curve is **ubiquitous** in data because it reflects the natural outcome of many **small, independent effects adding up** — and this is precisely what the **Central Limit Theorem** explains.

Let me know if you want to simulate this visually using Python!

```
[574]: # What about the absolute difference?
sns.histplot(data = all_sites['Rotten_diff'].apply(abs), kde = True)
```

```
[574]: <Axes: xlabel='Rotten_diff', ylabel='Count'>
```



```
[575]: all_sites['Rotten_diff'].apply(abs).describe()
```

```
[575]: count    146.000000
      mean     15.095890
      std     12.114015
      min       0.000000
      25%       6.000000
      50%      12.000000
      75%      21.000000
      max      66.000000
      Name: Rotten_diff, dtype: float64
```

0.2.10 What the Plot Tells You

From your description:

“I’m seeing a negative slope and more movies having Rotten difference closer to 0”

That means:

-
1. Most movies have small differences

- A peak near 0 means most critic and user ratings are **very close**.
 - This is a sign of **agreement** between critics and audiences in general.
-

2. Negative slope (curve drops off as the difference increases)

- Fewer movies have **large disagreements** (e.g., 30–40 point difference).
 - This shows that **agreement is common**, while **disagreement is less frequent**.
-

3. Right-skewed distribution

- A tail that extends to the right suggests:
 - While rare, some movies show **strong disagreement**.
 - These are outliers where either **critics loved it and users didn't**, or **users loved it and critics didn't**.
-

0.2.11 What This Tells You

- Most movies have **similar ratings** from critics and users.
- But a few show **significant disagreement**, indicating potential **polarizing content** or **bias**.
- This supports your earlier plot of `Rotten_diff`, but now you're focusing purely on the **magnitude** of disagreement, not the **direction** (who rated it higher).

Find out the top 5 movies having largest negative/positive difference:

```
[576]: all_sites.nsmallest(5, 'Rotten_diff')
```

```
[576]:
```

	FILM	RottenTomatoes	RottenTomatoes_User	\
3	Do You Believe? (2015)	18	84	
85	Little Boy (2015)	20	81	
105	Hitman: Agent 47 (2015)	7	49	
134	The Longest Ride (2015)	31	73	
125	The Wedding Ringer (2015)	27	66	

	Metacritic	Metacritic_User	IMDB	Metacritic_user_vote_count	\
3	22	4.7	5.4	31	
85	30	5.9	7.4	38	
105	28	3.3	5.9	67	
134	33	4.8	7.2	49	
125	35	3.3	6.7	126	

	IMDB_user_vote_count	Rotten_diff
3	3136	-66
85	5927	-61
105	4260	-42

134	25214	-42
125	37292	-39

```
[577]: # negative diff
print("Users love these movies, but critics hete these movies")
all_sites.nsmallest(5, 'Rotten_diff')[['FILM', 'Rotten_diff']]
```

Users love these movies, but critics hete these movies

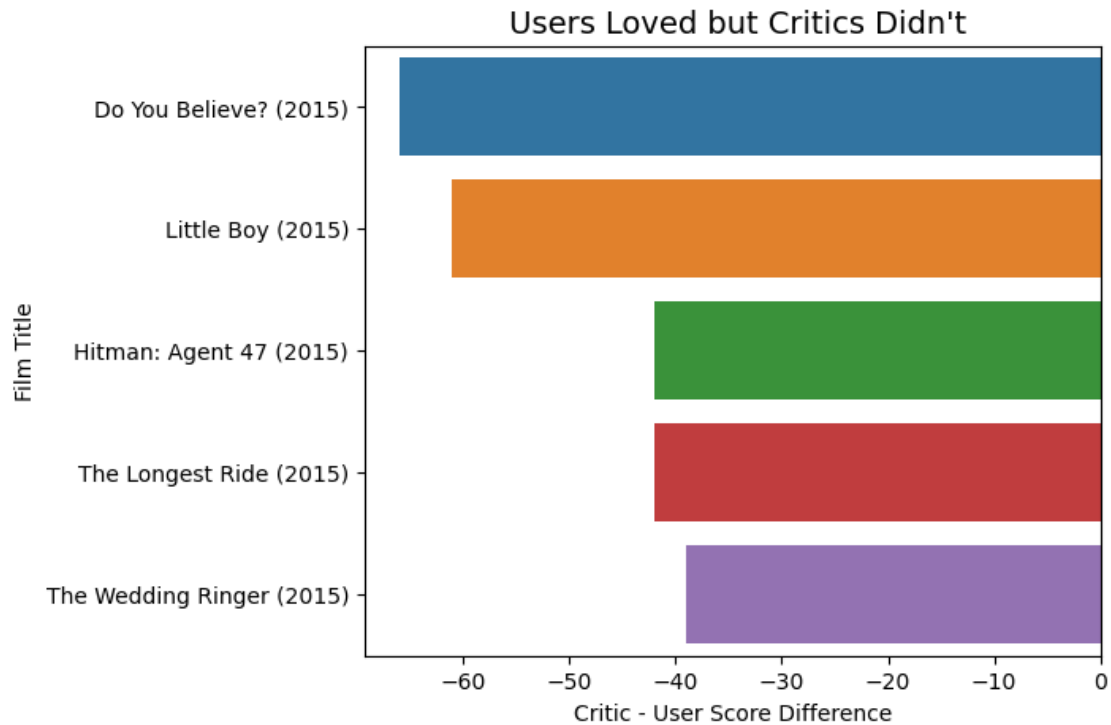
```
[577]:
```

	FILM	Rotten_diff
3	Do You Believe? (2015)	-66
85	Little Boy (2015)	-61
105	Hitman: Agent 47 (2015)	-42
134	The Longest Ride (2015)	-42
125	The Wedding Ringer (2015)	-39

```
[578]: # Get the 5 movies with the most negative difference
worst_diff = all_sites.nsmallest(5, 'Rotten_diff')[['FILM', 'Rotten_diff']]

# Set the plot style
plt.figure(figsize=(6,5))
sns.barplot(data=worst_diff, y='FILM', x='Rotten_diff', hue='FILM',
            ↪dodge=False, legend=False)

# Add titles and labels
plt.title(" Users Loved but Critics Didn't", fontsize=14)
plt.xlabel("Critic - User Score Difference")
plt.ylabel("Film Title")
plt.show()
```

```
[579]: # positive diff
all_sites.nlargest(5, 'Rotten_diff')[['FILM', 'Rotten_diff']]
```

```
[579]:
```

	FILM	Rotten_diff
69	Mr. Turner (2014)	42
112	It Follows (2015)	31
115	While We're Young (2015)	31
37	Welcome to Me (2015)	24
40	I'll See You In My Dreams (2015)	24

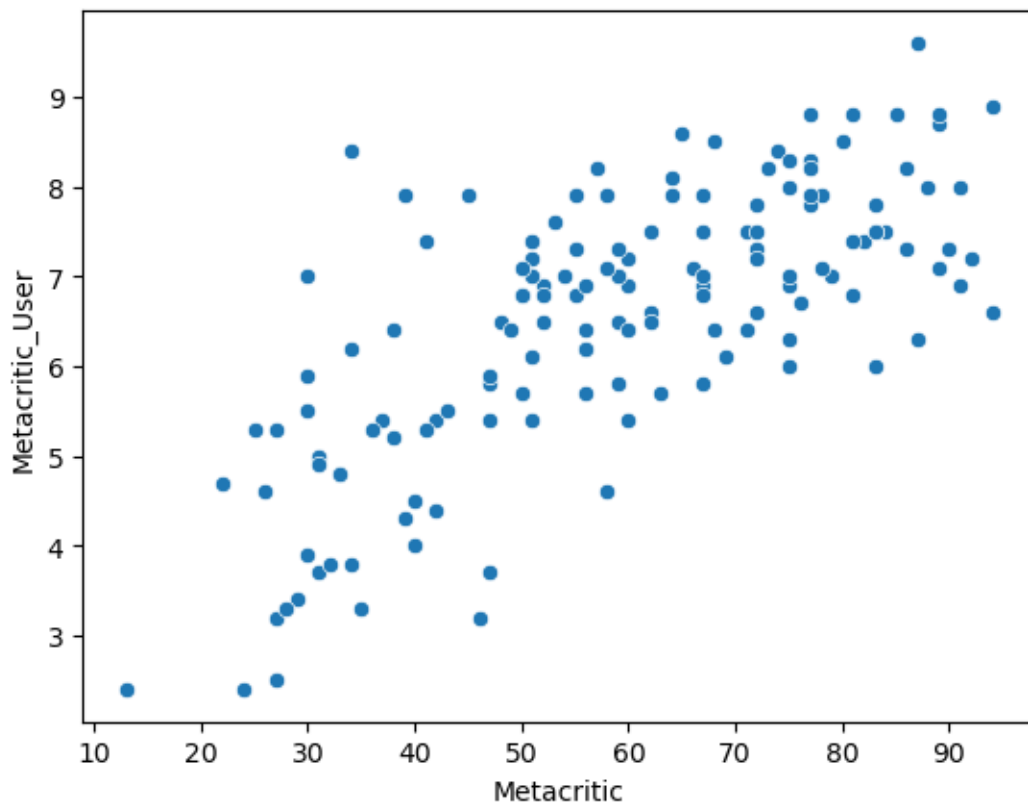
0.3 Metacritics

```
[580]: all_sites.columns
```

```
[580]: Index(['FILM', 'RottenTomatoes', 'RottenTomatoes_User', 'Metacritic',
        'Metacritic_User', 'IMDB', 'Metacritic_user_vote_count',
        'IMDB_user_vote_count', 'Rotten_diff'],
        dtype='object')
```

```
[581]: sns.scatterplot(data = all_sites, x = 'Metacritic', y = 'Metacritic_User')
```

```
[581]: <Axes: xlabel='Metacritic', ylabel='Metacritic_User'>
```



IMDB

```
[582]: all_sites.head()
```

```
[582]:
```

	FILM	RottenTomatoes	RottenTomatoes_User	\
0	Avengers: Age of Ultron (2015)	74	86	
1	Cinderella (2015)	85	80	
2	Ant-Man (2015)	80	90	
3	Do You Believe? (2015)	18	84	
4	Hot Tub Time Machine 2 (2015)	14	28	

	Metacritic	Metacritic_User	IMDB	Metacritic_user_vote_count	\
0	66	7.1	7.8	1330	
1	67	7.5	7.1	249	
2	64	8.1	7.8	627	
3	22	4.7	5.4	31	
4	29	3.4	5.1	88	

	IMDB_user_vote_count	Rotten_diff
0	271107	-12
1	65709	5

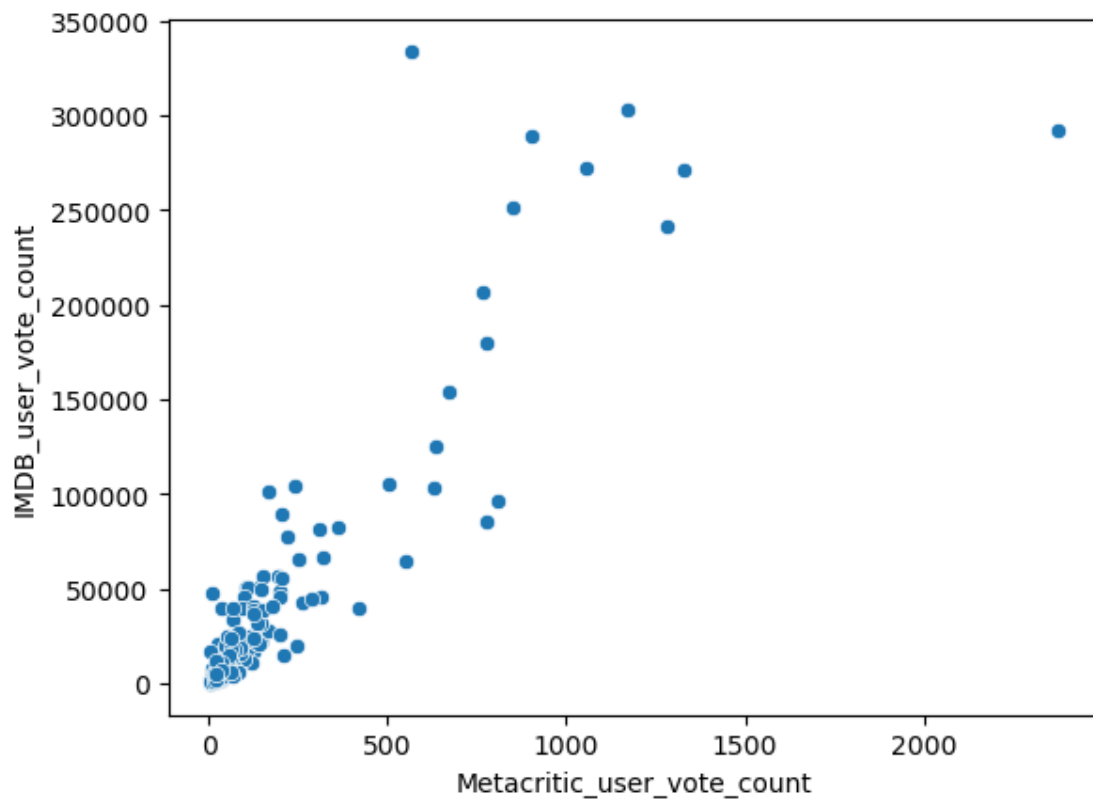
2	103660	-10
3	3136	-66
4	19560	-14

```
[583]: all_sites.columns
```

```
[583]: Index(['FILM', 'RottenTomatoes', 'RottenTomatoes_User', 'Metacritic',
        'Metacritic_User', 'IMDB', 'Metacritic_user_vote_count',
        'IMDB_user_vote_count', 'Rotten_diff'],
        dtype='object')
```

```
[584]: sns.scatterplot(data = all_sites, x = 'Metacritic_user_vote_count', y =
        ↪ 'IMDB_user_vote_count' )
```

```
[584]: <Axes: xlabel='Metacritic_user_vote_count', ylabel='IMDB_user_vote_count'>
```



```
[585]: all_sites.nlargest(1, 'IMDB_user_vote_count' )
```

```
[585]:
```

	FILM	RottenTomatoes	RottenTomatoes_User	\
14	The Imitation Game (2014)	90	92	

	Metacritic	Metacritic_User	IMDB	Metacritic_user_vote_count	\
14	73	8.2	8.1	566	

	IMDB_user_vote_count	Rotten_diff
14	334164	-2

```
[586]: all_sites.nlargest(1, 'Metacritic_user_vote_count' )
```

```
[586]:
```

	FILM	RottenTomatoes	RottenTomatoes_User	\
88	Mad Max: Fury Road (2015)	97	88	

	Metacritic	Metacritic_User	IMDB	Metacritic_user_vote_count	\
88	89	8.7	8.3	2375	

	IMDB_user_vote_count	Rotten_diff
88	292023	9

```
[587]: all_sites.head()
```

```
[587]:
```

	FILM	RottenTomatoes	RottenTomatoes_User	\
0	Avengers: Age of Ultron (2015)	74	86	
1	Cinderella (2015)	85	80	
2	Ant-Man (2015)	80	90	
3	Do You Believe? (2015)	18	84	
4	Hot Tub Time Machine 2 (2015)	14	28	

	Metacritic	Metacritic_User	IMDB	Metacritic_user_vote_count	\
0	66	7.1	7.8	1330	
1	67	7.5	7.1	249	
2	64	8.1	7.8	627	
3	22	4.7	5.4	31	
4	29	3.4	5.1	88	

	IMDB_user_vote_count	Rotten_diff
0	271107	-12
1	65709	5
2	103660	-10
3	3136	-66
4	19560	-14

0.4 Wheather or not fandango artifically boost the rating for better ticket sale??

```
[588]: df = pd.merge(fandango, all_sites, on = 'FILM', how = 'inner')
df
```

[588]:

	FILM	STARS	RATING	VOTES	YEAR	\
0	Fifty Shades of Grey (2015)	4.0	3.9	34846	2015	
1	Jurassic World (2015)	4.5	4.5	34390	2015	
2	American Sniper (2015)	5.0	4.8	34085	2015	
3	Furious 7 (2015)	5.0	4.8	33538	2015	
4	Inside Out (2015)	4.5	4.5	15749	2015	
..	...	...	...	...	...	
140	Kumiko, The Treasure Hunter (2015)	3.5	3.5	41	2015	
141	The Diary of a Teenage Girl (2015)	4.0	3.6	38	2015	
142	The Wrecking Crew (2015)	4.5	4.2	38	2015	
143	Tangerine (2015)	4.0	3.9	36	2015	
144	Maps to the Stars (2015)	3.5	3.1	35	2015	

	RottenTomatoes	RottenTomatoes_User	Metacritic	Metacritic_User	IMDB	\
0	25	42	46	3.2	4.2	
1	71	81	59	7.0	7.3	
2	72	85	72	6.6	7.4	
3	81	84	67	6.8	7.4	
4	98	90	94	8.9	8.6	
..	...	...	...	...	...	
140	87	63	68	6.4	6.7	
141	95	81	87	6.3	7.0	
142	93	84	67	7.0	7.8	
143	95	86	86	7.3	7.4	
144	60	46	67	5.8	6.3	

	Metacritic_user_vote_count	IMDB_user_vote_count	Rotten_diff
0	778	179506	-17
1	1281	241807	-10
2	850	251856	-13
3	764	207211	-3
4	807	96252	8
..	...	...	...
140	19	5289	24
141	18	1107	14
142	4	732	9
143	14	696	9
144	46	22440	14

[145 rows x 13 columns]

[589]: df.info()

```
<class 'pandas.core.frame.DataFrame'>
Int64Index: 145 entries, 0 to 144
Data columns (total 13 columns):
#   Column                                Non-Null Count  Dtype
#   ...
```

```

---  -----
0  FILM              145 non-null  object
1  STARS             145 non-null  float64
2  RATING            145 non-null  float64
3  VOTES             145 non-null  int64
4  YEAR              145 non-null  object
5  RottenTomatoes    145 non-null  int64
6  RottenTomatoes_User 145 non-null  int64
7  Metacritic        145 non-null  int64
8  Metacritic_User   145 non-null  float64
9  IMDB              145 non-null  float64
10 Metacritic_user_vote_count 145 non-null  int64
11 IMDB_user_vote_count 145 non-null  int64
12 Rotten_diff       145 non-null  int64
dtypes: float64(4), int64(7), object(2)
memory usage: 15.9+ KB

```

```
[590]: df.head()
```

```

[590]:
      FILM  STARS  RATING  VOTES  YEAR  RottenTomatoes  \
0  Fifty Shades of Grey (2015)    4.0    3.9   34846   2015         25
1      Jurassic World (2015)    4.5    4.5   34390   2015         71
2    American Sniper (2015)    5.0    4.8   34085   2015         72
3      Furious 7 (2015)    5.0    4.8   33538   2015         81
4      Inside Out (2015)    4.5    4.5   15749   2015         98

      RottenTomatoes_User  Metacritic  Metacritic_User  IMDB  \
0                42          46          3.2    4.2
1                81          59          7.0    7.3
2                85          72          6.6    7.4
3                84          67          6.8    7.4
4                90          94          8.9    8.6

      Metacritic_user_vote_count  IMDB_user_vote_count  Rotten_diff
0                778          179506          -17
1               1281          241807          -10
2                850          251856          -13
3                764          207211           -3
4                807          96252           8

```

```

[591]: # Normalize rating on same scale (say 5)
# RT
df['RT_Norm'] = round(df['RottenTomatoes']/20, 1)
df['RTU_Norm'] = round(df['RottenTomatoes_User']/20, 1)

```

```

[592]: # Metacritic
df['Meta_Norm'] = round(df['Metacritic']/20, 1)

```

```
df['Meta_U_Norm'] = round(df['Metacritic_User']/2, 1)
```

```
[593]: # IMDB
df['IMDB_Norm'] = round(df['IMDB']/2, 1)
```

```
[594]: df.head()
```

```
[594]:
```

	FILM	STARS	RATING	VOTES	YEAR	RottenTomatoes	\
0	Fifty Shades of Grey (2015)	4.0	3.9	34846	2015	25	
1	Jurassic World (2015)	4.5	4.5	34390	2015	71	
2	American Sniper (2015)	5.0	4.8	34085	2015	72	
3	Furious 7 (2015)	5.0	4.8	33538	2015	81	
4	Inside Out (2015)	4.5	4.5	15749	2015	98	

	RottenTomatoes_User	Metacritic	Metacritic_User	IMDB	\
0	42	46	3.2	4.2	
1	81	59	7.0	7.3	
2	85	72	6.6	7.4	
3	84	67	6.8	7.4	
4	90	94	8.9	8.6	

	Metacritic_user_vote_count	IMDB_user_vote_count	Rotten_diff	RT_Norm	\
0	778	179506	-17	1.2	
1	1281	241807	-10	3.6	
2	850	251856	-13	3.6	
3	764	207211	-3	4.0	
4	807	96252	8	4.9	

	RTU_Norm	Meta_Norm	Meta_U_Norm	IMDB_Norm
0	2.1	2.3	1.6	2.1
1	4.0	3.0	3.5	3.6
2	4.2	3.6	3.3	3.7
3	4.2	3.4	3.4	3.7
4	4.5	4.7	4.4	4.3

```
[595]: df.columns
```

```
[595]: Index(['FILM', 'STARS', 'RATING', 'VOTES', 'YEAR', 'RottenTomatoes',
        'RottenTomatoes_User', 'Metacritic', 'Metacritic_User', 'IMDB',
        'Metacritic_user_vote_count', 'IMDB_user_vote_count', 'Rotten_diff',
        'RT_Norm', 'RTU_Norm', 'Meta_Norm', 'Meta_U_Norm', 'IMDB_Norm'],
        dtype='object')
```

```
[596]: df[['STARS', 'RATING', 'RT_Norm', 'RTU_Norm', 'Meta_Norm', 'Meta_U_Norm',
        ↪ 'IMDB_Norm']]
```

```
[596]:
```

	STARS	RATING	RT_Norm	RTU_Norm	Meta_Norm	Meta_U_Norm	IMDB_Norm
0	4.0	3.9	1.2	2.1	2.3	1.6	2.1
1	4.5	4.5	3.6	4.0	3.0	3.5	3.6
2	5.0	4.8	3.6	4.2	3.6	3.3	3.7
3	5.0	4.8	4.0	4.2	3.4	3.4	3.7
4	4.5	4.5	4.9	4.5	4.7	4.4	4.3
..	...	...	...	...	...	...	...
140	3.5	3.5	4.4	3.2	3.4	3.2	3.4
141	4.0	3.6	4.8	4.0	4.4	3.2	3.5
142	4.5	4.2	4.6	4.2	3.4	3.5	3.9
143	4.0	3.9	4.8	4.3	4.3	3.6	3.7
144	3.5	3.1	3.0	2.3	3.4	2.9	3.2

[145 rows x 7 columns]

```
[597]: norm_scores = df[['STARS', 'RATING', 'RT_Norm', 'RTU_Norm', 'Meta_Norm', 'Meta_U_Norm', 'IMDB_Norm']]
norm_scores
```

```
[597]:
```

	STARS	RATING	RT_Norm	RTU_Norm	Meta_Norm	Meta_U_Norm	IMDB_Norm
0	4.0	3.9	1.2	2.1	2.3	1.6	2.1
1	4.5	4.5	3.6	4.0	3.0	3.5	3.6
2	5.0	4.8	3.6	4.2	3.6	3.3	3.7
3	5.0	4.8	4.0	4.2	3.4	3.4	3.7
4	4.5	4.5	4.9	4.5	4.7	4.4	4.3
..	...	...	...	...	...	...	...
140	3.5	3.5	4.4	3.2	3.4	3.2	3.4
141	4.0	3.6	4.8	4.0	4.4	3.2	3.5
142	4.5	4.2	4.6	4.2	3.4	3.5	3.9
143	4.0	3.9	4.8	4.3	4.3	3.6	3.7
144	3.5	3.1	3.0	2.3	3.4	2.9	3.2

[145 rows x 7 columns]

0.4.1 Compare Distribution of Scores across the sites.

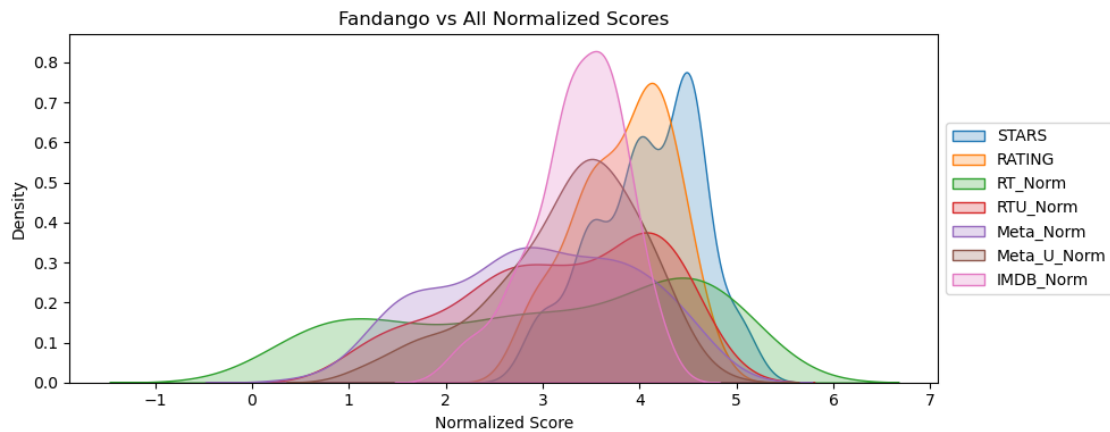
```
[598]: plt.figure(figsize=(10, 4))

for name in norm_scores.columns:
    sns.kdeplot(data=norm_scores, x=name, label=name, fill=True)

plt.title("Fandango vs All Normalized Scores")
plt.xlabel("Normalized Score")
plt.ylabel("Density")
plt.legend(loc='center left', bbox_to_anchor=(1, 0.5)) # Put legend outside
plot
plt.tight_layout()
```



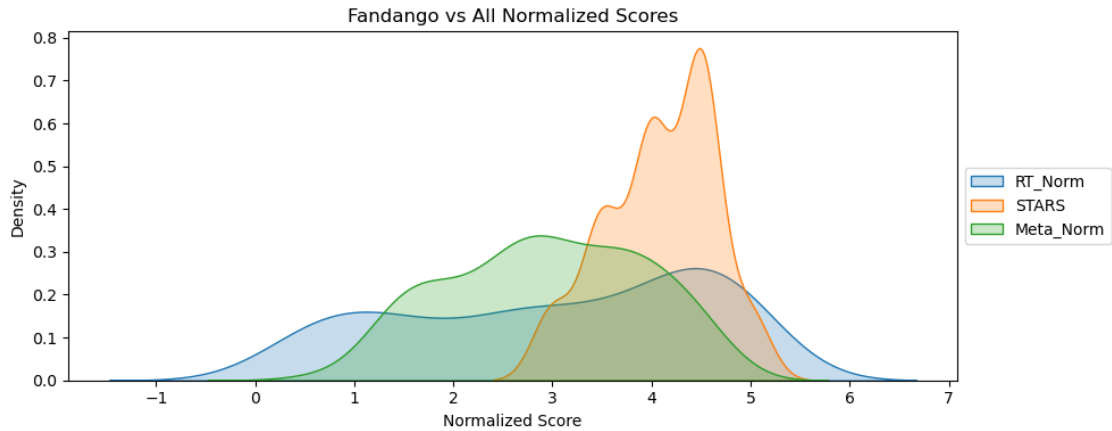
```
plt.show()
```



```
[599]: norm_scores.columns
```

```
[599]: Index(['STARS', 'RATING', 'RT_Norm', 'RTU_Norm', 'Meta_Norm', 'Meta_U_Norm',  
          'IMDB_Norm'],  
          dtype='object')
```

```
[600]: plt.figure(figsize=(10, 4))  
  
for name in ['RT_Norm', 'STARS', 'Meta_Norm']:  
    sns.kdeplot(data=norm_scores, x=name, label=name, fill=True)  
  
plt.title("Fandango vs All Normalized Scores")  
plt.xlabel("Normalized Score")  
plt.ylabel("Density")  
plt.legend(loc='center left', bbox_to_anchor=(1, 0.5)) # Put legend outside_  
↳ plot  
plt.tight_layout()  
plt.show()
```



0.4.2 Clustermap Explanation

Input:

- `norm_scores` — a DataFrame where:
 - **Each column** is a rating source (e.g., Fandango, IMDB, Metacritic, RottenTomatoes).
 - **Each row** represents a movie with **normalized ratings** from each source.

clustermap:

- Performs **hierarchical clustering** and displays a **heatmap**.
- Groups together **movies with similar rating patterns** across platforms.

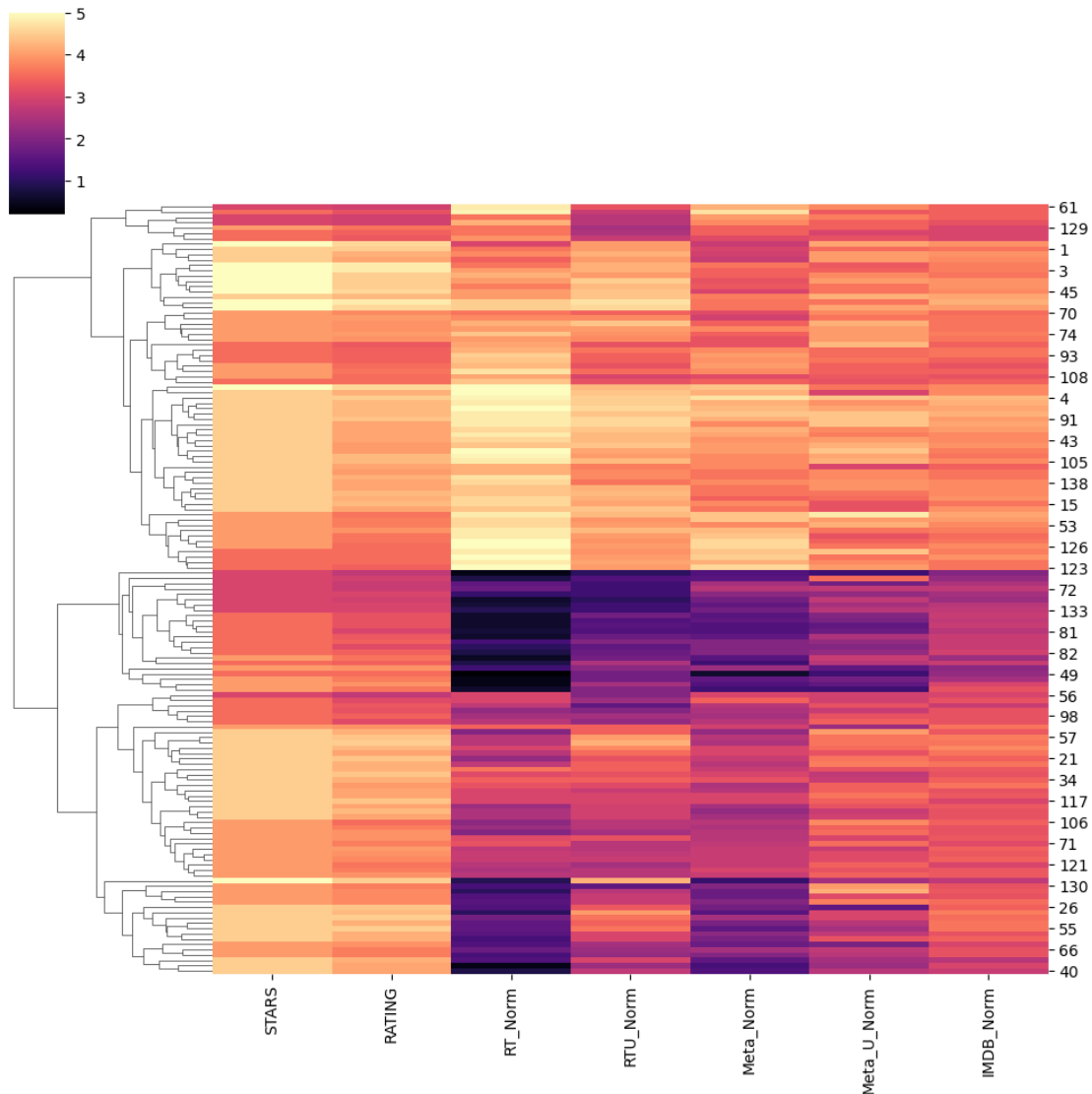
`cmap='magma':`

- Applies the **Magma color scale**:
 - **Dark** = low normalized score
 - **Bright** = high normalized score

`col_cluster=False:`

- Disables clustering of columns.
- Only **rows (movies)** are clustered based on similarity in rating patterns.
- Column order remains fixed (useful if you want to preserve rating source order).

```
[601]: # Cluster
sns.clustermap(norm_scores,cmap = 'magma',col_cluster= False)
plt.show()
```



- **Fandango** column is mostly bright-colored (values between 4–5), which means:
 - Most movies were rated **very highly**.
 - This looks **suspicious** and may indicate **inflated or biased ratings**.
- **Other platforms** show a mix of colors across movies:
 - Suggests **balanced, varied ratings** — some movies are liked, others not.
 - This is **expected and more trustworthy**.
- This visual evidence supports claims that **Fandango displayed overly optimistic ratings**, compared to more neutral platforms like IMDB or Metacritic.

```
[602]: norm_film = df[['FILM', 'STARS', 'RATING', 'RT_Norm', 'RTU_Norm', 'Meta_Norm', 'Meta_U_Norm', 'IMDB_Norm']]
norm_film
```

```
[602]:
```

	FILM	STARS	RATING	RT_Norm	RTU_Norm	\
0	Fifty Shades of Grey (2015)	4.0	3.9	1.2	2.1	
1	Jurassic World (2015)	4.5	4.5	3.6	4.0	
2	American Sniper (2015)	5.0	4.8	3.6	4.2	
3	Furious 7 (2015)	5.0	4.8	4.0	4.2	
4	Inside Out (2015)	4.5	4.5	4.9	4.5	
..	...	...	...	...	...	
140	Kumiko, The Treasure Hunter (2015)	3.5	3.5	4.4	3.2	
141	The Diary of a Teenage Girl (2015)	4.0	3.6	4.8	4.0	
142	The Wrecking Crew (2015)	4.5	4.2	4.6	4.2	
143	Tangerine (2015)	4.0	3.9	4.8	4.3	
144	Maps to the Stars (2015)	3.5	3.1	3.0	2.3	

	Meta_Norm	Meta_U_Norm	IMDB_Norm
0	2.3	1.6	2.1
1	3.0	3.5	3.6
2	3.6	3.3	3.7
3	3.4	3.4	3.7
4	4.7	4.4	4.3
..	...	...	...
140	3.4	3.2	3.4
141	4.4	3.2	3.5
142	3.4	3.5	3.9
143	4.3	3.6	3.7
144	3.4	2.9	3.2

[145 rows x 8 columns]

```
[603]: norm_film.nsmallest(10,'RT_Norm')
```

```
[603]:
```

	FILM	STARS	RATING	RT_Norm	RTU_Norm	\
49	Paul Blart: Mall Cop 2 (2015)	3.5	3.5	0.2	1.8	
25	Taken 3 (2015)	4.5	4.1	0.4	2.3	
28	Fantastic Four (2015)	3.0	2.7	0.4	1.0	
54	Hot Pursuit (2015)	4.0	3.7	0.4	1.8	
84	Hitman: Agent 47 (2015)	4.0	3.9	0.4	2.4	
50	The Boy Next Door (2015)	4.0	3.6	0.5	1.8	
77	Seventh Son (2015)	3.5	3.2	0.6	1.8	
78	Mortdecai (2015)	3.5	3.2	0.6	1.5	
83	Sinister 2 (2015)	3.5	3.3	0.6	1.7	
87	Unfinished Business (2015)	3.5	3.2	0.6	1.4	

	Meta_Norm	Meta_U_Norm	IMDB_Norm
49	0.6	1.2	2.2
25	1.3	2.3	3.0
28	1.4	1.2	2.0
54	1.6	1.8	2.4

84	1.4	1.6	3.0
50	1.5	2.8	2.3
77	1.5	2.0	2.8
78	1.4	1.6	2.8
83	1.6	2.5	2.8
87	1.6	1.9	2.7

- If u see this Taken 3 movie:

```
[604]: # Using Matplotlib

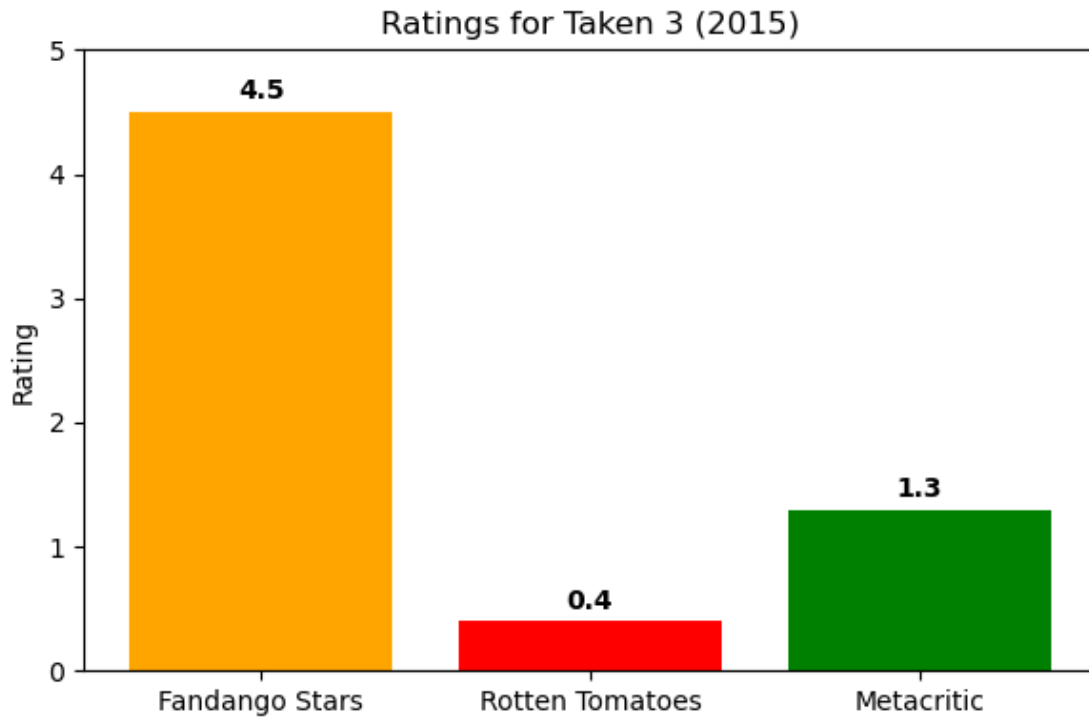
# Filter just the Taken 3 movie
taken3 = norm_film[norm_film['FILM'] == 'Taken 3 (2015)']

# Prepare data for plotting
ratings = {
    'Fandango Stars': taken3['STARS'].values[0],
    'Rotten Tomatoes': taken3['RT_Norm'].values[0],
    'Metacritic': taken3['Meta_Norm'].values[0]
}

# Plot
plt.figure(figsize=(6, 4))
plt.bar(ratings.keys(), ratings.values(), color=['orange', 'red', 'green'])
plt.title("Ratings for Taken 3 (2015)")
plt.ylabel("Rating")
plt.ylim(0, 5)

# Add value labels
for i, value in enumerate(ratings.values()):
    plt.text(i, value + 0.1, f"{value:.1f}", ha='center', fontweight='bold')

plt.tight_layout()
plt.show()
```



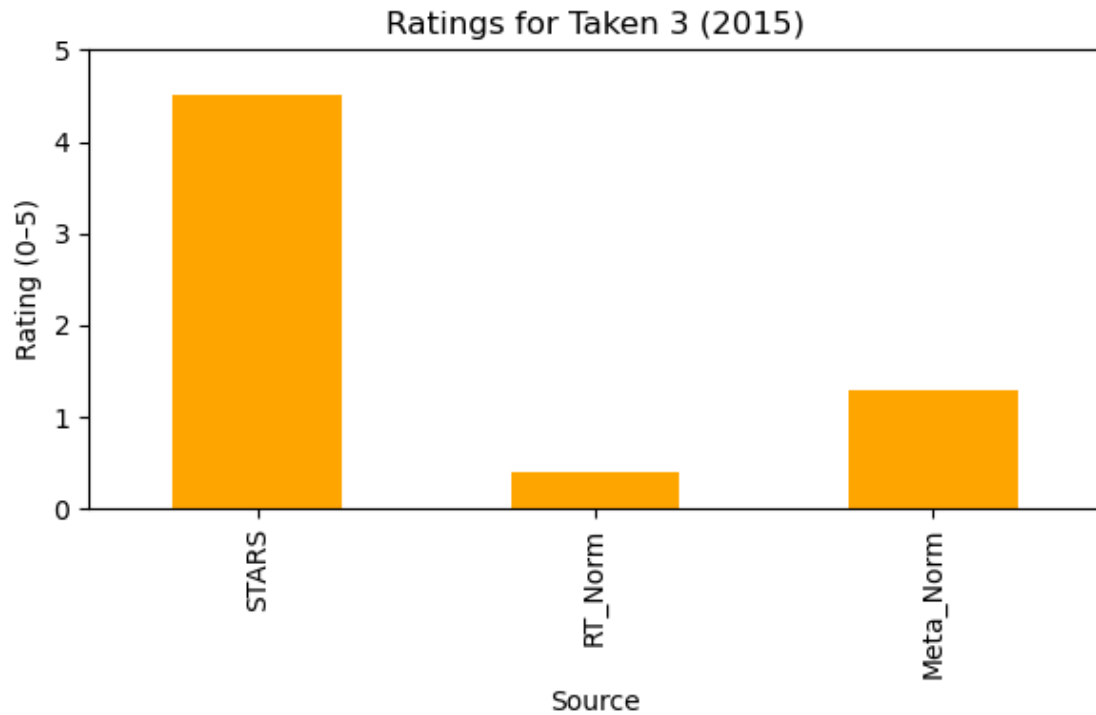
```
[605]: taken3
```

```
[605]:
```

	FILM	STARS	RATING	RT_Norm	RTU_Norm	Meta_Norm	Meta_U_Norm	\
25	Taken 3 (2015)	4.5	4.1	0.4	2.3	1.3	2.3	
	IMDB_Norm							
25	3.0							

```
[606]: # Using pandas
# Select only the columns you want to plot
plot_data = taken3[['STARS', 'RT_Norm', 'Meta_Norm']].melt(var_name='Source', \
    value_name='Rating')

# Plot using Pandas
plot_data.set_index('Source').plot(kind='bar', legend=False, figsize=(6, 4), \
    color=['orange', 'red', 'green'])
plt.title("Ratings for Taken 3 (2015)")
plt.ylabel("Rating (0-5)")
plt.ylim(0, 5)
plt.tight_layout()
plt.show()
```

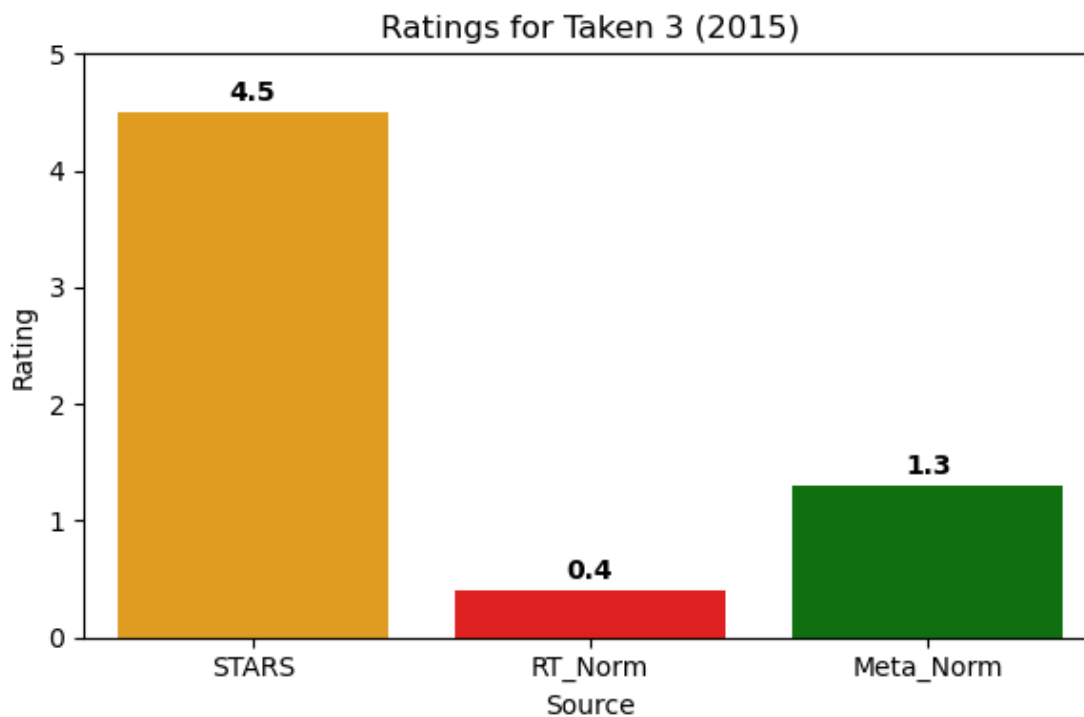


```
[607]: # Using seaborn
# Same reshaped data
plot_data = taken3[['STARS', 'RT_Norm', 'Meta_Norm']].melt(var_name='Source',
    ↳value_name='Rating')

# Plot using Seaborn
plt.figure(figsize=(6, 4))
sns.barplot(data=plot_data, x='Source', y='Rating', palette=['orange', 'red',
    ↳'green'])
plt.title("Ratings for Taken 3 (2015)")
plt.ylim(0, 5)

# Add value labels
for i, row in plot_data.iterrows():
    plt.text(i, row['Rating'] + 0.1, f"{row['Rating']:.1f}", ha='center',
    ↳fontweight='bold')

plt.tight_layout()
plt.show()
```



```
[608]: norm_scores
```

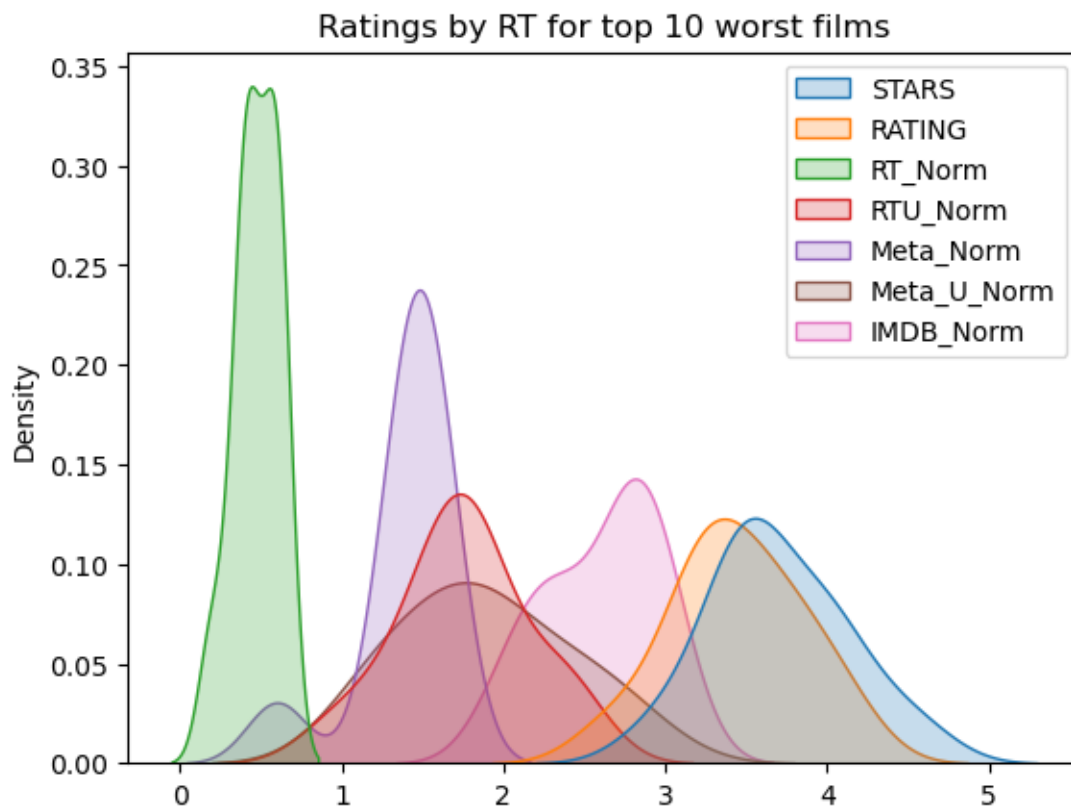
```
[608]:
```

	STARS	RATING	RT_Norm	RTU_Norm	Meta_Norm	Meta_U_Norm	IMDB_Norm
0	4.0	3.9	1.2	2.1	2.3	1.6	2.1
1	4.5	4.5	3.6	4.0	3.0	3.5	3.6
2	5.0	4.8	3.6	4.2	3.6	3.3	3.7
3	5.0	4.8	4.0	4.2	3.4	3.4	3.7
4	4.5	4.5	4.9	4.5	4.7	4.4	4.3
..	...	...	...	...	...	...	...
140	3.5	3.5	4.4	3.2	3.4	3.2	3.4
141	4.0	3.6	4.8	4.0	4.4	3.2	3.5
142	4.5	4.2	4.6	4.2	3.4	3.5	3.9
143	4.0	3.9	4.8	4.3	4.3	3.6	3.7
144	3.5	3.1	3.0	2.3	3.4	2.9	3.2

```
[145 rows x 7 columns]
```

```
[609]: sns.kdeplot(data = norm_scores.nsmallest(10,'RT_Norm'), fill = True)
plt.title('Ratings by RT for top 10 worst films' )
```

```
[609]: Text(0.5, 1.0, 'Ratings by RT for top 10 worst films')
```

Observation * For those worst 10 movies, Metacritics gave intermediate ratings. * For those worst 10 movies, this fandango gave very high ratings.

0.4.3 For taken 3 movie

```
[610]: norm_film.iloc[25]
```

```
[610]: FILM          Taken 3 (2015)
STARS              4.5
RATING             4.1
RT_Norm            0.4
RTU_Norm           2.3
Meta_Norm          1.3
Meta_U_Norm        2.3
IMDB_Norm          3.0
Name: 25, dtype: object
```

```
[ ]:
```

```
[ ]:
```

[]:

[]:

[]:

[]:

[]: